TECHNICAL REFERENCE GUIDE

PostgreSQL pg_rewind vs. pg_basebackup

Rebuilding an Old Primary as a Standby After Failover

Validated against PostgreSQL 18 official documentation

Scope

Timeline divergence • WAL requirements • replication slots • safe recovery workflow

Core conclusion For a healthy old primary from the same cluster history, pg_rewind can be far faster than a full pg_basebackup. It is safe only when its prerequisites and required WAL history are available.

Contents

1. Executive Summary

After a PostgreSQL standby is promoted, the old primary cannot normally be restarted as a standby without first being synchronized. The promoted server begins a new timeline, while the old primary retains data from the former timeline. pg_rewind is designed to bring the old primary back to the new primary's state by repairing that divergence.

- Use pg_rewind when the target is a stopped, healthy copy of the same PostgreSQL cluster and the required history is available.
- Use pg_basebackup when the target is new, corrupted, unrelated, missing prerequisites, or cannot be rewound safely.
- Do not treat a replication slot as a replacement for WAL archiving or for pg_rewind's divergence-WAL requirements.

> **Technical correction** pg_rewind does not literally copy only changed blocks. It copies changed blocks in existing relation files and may also copy complete new relation files, WAL segments, configuration files, and other required files.

2. Failover and Timeline Divergence

2.1 Normal state before failure

```
PG01 (Primary)  ── streaming replication ──>  PG02 (Standby)
```

Both servers share the same cluster history. PG02 continuously receives and replays WAL records generated by PG01.

2.2 State after promotion

```
PG01 (Old Primary, stopped)

PG02 (Promoted New Primary)  ── starts a new timeline
```

Promotion creates a new timeline on PG02. If PG01 later returns, its local data directory still reflects the old timeline. Starting it without repair can create split-brain risk or recovery failure.

2.3 Why a simple restart is unsafe

- The servers may contain different changes after their common checkpoint.
- The old primary may contain blocks that do not match the promoted server.
- The new primary is now the authoritative source for the cluster.

3. How pg_rewind Works

pg_rewind compares the target data directory with the source cluster, finds the point where their timelines diverged, identifies blocks changed on the target after that point, and copies the data required to make the target consistent with the source.

1. Find the common history.

The tool reads timeline history and control information to locate the last common checkpoint.

2. Scan target WAL.

It processes the target's WAL from the divergence point to determine which data blocks were changed on the target.

3. Synchronize files.

Changed blocks in existing relation files are copied from the source. New and other required files may be copied in full.

4. Complete recovery.

The rewound target must start in recovery and replay WAL until it becomes consistent with the source.

> **Performance point** The benefit depends on how much data changed after divergence. A 5 TB cluster does not automatically mean a 5 TB rewind; however, a long outage or heavy write workload can increase the copied data and recovery time.

4. Requirements and Prechecks

Requirement	What to verify	Why it matters
Same cluster ancestry	Source and target originated from the same PostgreSQL cluster.	pg_rewind repairs divergence; it does not clone an unrelated cluster.
Clean target shutdown	The target PostgreSQL instance is stopped cleanly.	The target data directory must not change during rewind.
Checksums or wal_log_hints	Data checksums enabled, or wal_log_hints = on.	Required to identify changed pages safely.
full_page_writes	full_page_writes = on.	Required by pg_rewind.
WAL history	Required WAL exists locally or in a usable archive.	Needed to locate and process divergence.
Source access	Local source directory or SQL connection with suitable privileges.	Needed to read source files and metadata.

4.1 Configuration checks

```
SHOW wal_log_hints;
SHOW full_page_writes;
SHOW data_checksums;
```

For data checksums, SHOW data_checksums is available in supported modern PostgreSQL versions. At the operating-system level, pg_controldata can also confirm the checksum version:

```
pg_controldata "$PGDATA" | grep -i "Data page checksum version"
```

4.2 Dry-run before writing

```
pg_rewind \
  --target-pgdata=/var/lib/postgresql/18/main \
  --source-server="host=<NEW_PRIMARY> port=5432 user=<REWIND_USER> dbname=postgres" \
  --dry-run --progress
```

Important A dry run checks feasibility but does not replace a backup or full operational validation.

5. WAL Availability and Replication Slots

5.1 WAL required by pg_rewind

pg_rewind needs enough target-side WAL history to determine what changed after the divergence point. When necessary WAL is no longer present in pg_wal, the -c / --restore-target-wal option can retrieve it through the target's configured restore_command.

```
pg_rewind -c \
  --target-pgdata=/var/lib/postgresql/18/main \
  --source-server="host=<NEW_PRIMARY> port=5432 user=<REWIND_USER> dbname=postgres"
```

5.2 What a physical replication slot does

A physical replication slot tells the upstream primary to retain WAL needed by a replication consumer. After PG01 is rebuilt as a standby, a slot on PG02 can help prevent required streaming WAL from being removed before PG01 receives it.

```
SELECT * FROM pg_create_physical_replication_slot('pg01_slot');
```

5.3 What a slot does not guarantee

Do not confuse the roles A slot protects WAL on the current upstream server for its consumer. pg_rewind also depends on timeline and target WAL history around the divergence point. Maintain a tested WAL archive when recovery design requires older WAL.

5.4 Disk-space risk

If a standby remains offline and its slot cannot advance, the primary can retain large amounts of WAL. This can fill the filesystem that contains pg_wal. Set a reasonable max_slot_wal_keep_size according to the recovery design and monitor slot health.

```
SHOW max_slot_wal_keep_size;

SELECT slot_name, slot_type, active, restart_lsn,
       wal_status, safe_wal_size, invalidation_reason
FROM pg_replication_slots
ORDER BY slot_name;
```

6. Recovery Workflow

Assumption: PG02 has been promoted and is the authoritative primary. PG01 is the old primary that must rejoin as a standby.

1. Protect the new primary

Confirm PG02 accepts writes, applications use the correct endpoint, and fencing prevents PG01 from accepting client writes.

2. Stop PG01

Ensure PostgreSQL is fully stopped. Do not run pg_rewind against a live target data directory.

3. Preserve evidence and fallback

Retain logs and, where practical, a recoverable copy or snapshot of PG01 before changing its data directory.

4. Confirm ancestry and prerequisites

Check cluster identity, PostgreSQL major version, checksums or wal_log_hints, full_page_writes, WAL availability, storage, and permissions.

5. Run a dry run

Use --dry-run and review all messages before running the actual rewind.

6. Execute pg_rewind

Use PG02 as the source and PG01 as the target. Add -R to write standby recovery configuration.

7. Review recovery configuration

Verify standby.signal, primary_conninfo, primary_slot_name if used, restore_command, permissions, and secrets handling.

8. Start PG01 as a standby

Monitor the PostgreSQL log for recovery, timeline, restore, and streaming messages.

9. Validate catch-up

Confirm PG01 is in recovery, PG02 sees the streaming connection, and replay lag reaches the accepted threshold.

10. Re-enable operational controls

Restore monitoring, backup coverage, alerting, and documented failover readiness.

7. pg_rewind Command and Validation

7.1 Representative command

```
pg_rewind \
  --target-pgdata=/var/lib/postgresql/18/main \
  --source-server="host=<NEW_PRIMARY> port=5432 user=<REWIND_USER> dbname=postgres sslmode=require" \
  --progress \
  --write-recovery-conf
```

Option summary:

Option	Purpose
--target-pgdata	Path to the stopped old primary's data directory.
--source-server	Connection string for the new primary/source cluster.
--progress	Displays progress while files are processed.
--write-recovery-conf / -R	Creates standby.signal and appends connection settings to postgresql.auto.conf.
--restore-target-wal / -c	Allows required target WAL to be restored through restore_command.
--dry-run	Checks the operation without modifying the target.

7.2 Recovery settings to review

```
primary_conninfo = 'host=<NEW_PRIMARY> port=5432 user=<REPLICATION_USER> sslmode=require'
primary_slot_name = 'pg01_slot'
restore_command = '<tested WAL restore command>'
```

Avoid placing plaintext passwords directly in commands or world-readable configuration files. Use a protected .pgpass file or an approved secret-management method.

7.3 Validation queries

```
-- Run on PG01, expected result: true
SELECT pg_is_in_recovery();

-- Run on PG02
SELECT application_name, client_addr, state, sync_state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       pg_wal_lsn_diff(sent_lsn, replay_lsn) AS byte_lag
FROM pg_stat_replication;
```

7.4 Log and functional checks

- Confirm no unexpected promotion, PANIC, invalid checkpoint, missing WAL, or permission errors.
- Confirm the standby follows the expected timeline and reaches streaming state.
- Perform an application-safe read validation on the standby only if hot_standby is enabled and policy permits it.

- Confirm backup tooling and monitoring identify PG02 as primary and PG01 as standby.

8. When to Use pg_basebackup

Use a fresh base backup when rewind is not supported, cannot be proven safe, or the target cannot be trusted. pg_basebackup creates a new copy of the entire cluster and can write the recovery configuration for a standby.

- Checksums are disabled and wal_log_hints was not enabled before the divergence.
- Required target WAL cannot be found locally or in the WAL archive.
- The old primary has storage corruption, incomplete files, or questionable filesystem integrity.
- Source and target are not copies of the same cluster history.
- pg_rewind fails after it begins modifying the target. PostgreSQL warns that the target may no longer be recoverable; a new backup is then recommended.

```
pg_basebackup \
  -h <NEW_PRIMARY> \
  -p 5432 \
  -U <REPLICATION_USER> \
  -D /var/lib/postgresql/18/main \
  -P -R -X stream
```

Capacity planning A full base backup transfers the cluster data plus required WAL. Duration depends on cluster size, network, source and target storage throughput, compression, concurrent workload, checkpoints, and rate limits.

9. Comparison and Decision Guide

Area	pg_rewind	pg_basebackup
Purpose	Repair a diverged copy of the same cluster.	Create a fresh physical copy of a cluster.
Typical failover use	Return the old primary as a standby.	Rebuild a standby from scratch.
Transfer volume	Changed blocks plus other required files.	Full cluster copy for a normal full base backup.
Speed on multi-TB systems	Often faster when divergence is small.	Usually slower because the full cluster is copied.
Preconditions	Same ancestry; clean target; checksums or wal_log_hints; full_page_writes; required WAL.	Replication access, capacity, permissions, and WAL/backup requirements.
Damaged target	Not preferred.	Preferred fresh rebuild.
Failure handling	A failed run may leave target unusable.	Restart/rebuild according to the backup procedure.

9.1 Fast decision guide

Question	If yes	If no
Same cluster history?	Continue checks.	Use pg_basebackup.
Target cleanly stopped and storage healthy?	Continue checks.	Repair storage or use pg_basebackup.
Checksums enabled or wal_log_hints was on?	Continue checks.	Use pg_basebackup.
Required WAL available locally or from archive?	Dry-run pg_rewind.	Use pg_basebackup.
Dry run succeeds and result is operationally acceptable?	Run pg_rewind with fallback ready.	Use pg_basebackup.

10. Operational Best Practices

10.1 Prepare before the incident

- Enable data checksums when the cluster is initialized, or set wal_log_hints = on in advance if pg_rewind is part of the HA design.
- Keep full_page_writes enabled.
- Configure and routinely test WAL archiving and restore_command.
- Document fencing so the former primary cannot accept writes after failover.
- Test pg_rewind and pg_basebackup paths during planned exercises, including the fallback decision.

10.2 Monitor continuously

- Replication lag and pg_stat_replication state.
- Replication slot activity, retained WAL, wal_status, safe_wal_size, and invalidation_reason.
- pg_wal filesystem usage and archive success/failure.
- Backup age, restore-test results, and recovery time against RTO/RPO targets.

10.3 Security and change control

- Use a dedicated least-privilege rewind/replication role and encrypted connections.
- Protect .pgpass and configuration files with appropriate operating-system permissions.
- Record source, target, timeline, command output, start/end time, copied volume, validation results, and approver.
- Do not delete the old data directory until the rebuilt standby and fallback coverage are verified.

11. Official PostgreSQL References

1. PostgreSQL 18 - pg_rewind
   https://www.postgresql.org/docs/18/app-pgrewind.html
2. PostgreSQL 18 - pg_basebackup
   https://www.postgresql.org/docs/18/app-pgbasebackup.html
3. PostgreSQL 18 - Log-Shipping Standby Servers
   https://www.postgresql.org/docs/18/warm-standby.html
4. PostgreSQL 18 - Replication Configuration
   https://www.postgresql.org/docs/18/runtime-config-replication.html
5. PostgreSQL 18 - pg_replication_slots View
   https://www.postgresql.org/docs/18/view-pg-replication-slots.html

Validation basis:

PostgreSQL 18 documentation.

Always use the documentation matching the installed major version and apply the latest supported minor release for that major version.